



AUDIT REPORT

April 2026

For



REAL LIBERTY COPPER

Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
 Medium Severity Issues	14
1. Chainlink Oracle Returns Incorrect Price When Circuit Breaker Bounds Are Hit	14
 Low Severity Issues	15
2. Missing Blacklist Mechanism for Restricted Addresses	15
3. ETH Silently Absorbed on ERC20 Payment in buyToken()	16
4. Missing Expiry (Deadline) in Signature Verification	17
 Informational Issues	18
5. Single-Step Ownership Transfer Allows Permanent Loss of Contract Control	18
6. Excessively Lenient Oracle Staleness Threshold Accepts Outdated Prices	19
7. Missing pending-request guard lets a later owner overwrite an existing redemption request	20
Functional Tests	21
Automated Tests	21
Threat Model	22
Closing Summary & Disclaimer	40



Executive Summary

Project Name	Liberty Protocol
Protocol Type	RWA
Project URL	https://reallibertycopper.com/
Overview	LibertyICO is a token sale contract that sells LibertyCoin in exchange for supported payment assets like ETH and configured ERC20 tokens. It uses Chainlink price feeds to convert the incoming payment amount into the correct amount of LibertyCoin. Each purchase must be approved by an off-chain signer through an EIP-712 signature, which helps control who can buy and under what terms. The contract includes admin controls to pause sales, update payment-token settings, change pricing parameters, and recover collected funds. Overall, it acts as a controlled on-chain sale gateway for distributing LibertyCoin based on verified payments and oracle pricing.
Audit Scope	The scope of this Audit was to analyze the Liberty Protocol Smart Contracts for quality, security, and correctness.
Source Code link	LibertyCoin: https://sepolia.etherscan.io/address/0x002B820D0Ef8546DC1aA0DAF3FA5B2d3661329F9 LibertyNft: https://sepolia.etherscan.io/address/0xF4e5BA6F7b006e9C649C168bFFe1Efc74Ed782d9 LibertyOracle: https://sepolia.etherscan.io/address/0x1EcD7302ef91D690E1cBbF8F491bE677Fe7dcb22 LibertyICO : https://sepolia.etherscan.io/address/0x51De0252C107847914e26027E5CD5d93c8B77a8B



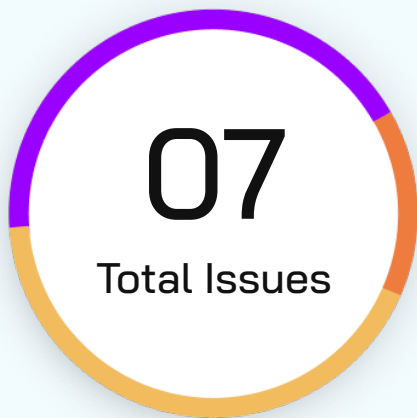
Contracts in Scope	contracts/LibertyOracle.sol contracts/LibertyNFT.sol contracts/LibertyICO.sol contracts/LibertyCoin.sol
Commit Hash	39524073dfed17ef6ac6d013bd5b5137fae814a0
Language	Solidity
Blockchain	Ethereum
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	2nd April 2026 - 8th April 2026
Updated Code Received	20th April 2026
Review 2	21st April 2026
Fixed In	LibertyCoin: https://sepolia.etherscan.io/address/0xbe757045d10926b603fce768dcf91ea16e474b45#code LibertyNft: https://sepolia.etherscan.io/address/0x0149e163a13dbcf694dc1c60008de4c81347fa60#code LibertyOracle: https://sepolia.etherscan.io/address/0x6a06eafde7c5b309444f97746d27c11162d592ad#code LibertyICO : https://sepolia.etherscan.io/address/0xcc73c2cb666f080c7b1d4db344c1ea8680175af7#code

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0.0%)
High	0 (0.0%)
Medium	1 (14.28%)
Low	3 (42.85%)
Informational	3 (42.85%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	1
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	1	3	2



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Chainlink Oracle Returns Incorrect Price When Circuit Breaker Bounds Are Hit	Medium	Resolved
2	Missing Blacklist Mechanism for Restricted Addresses	Low	Resolved
3	ETH Silently Absorbed on ERC20 Payment in buyToken()	Low	Resolved
4	Missing Expiry (Deadline) in Signature Verification	Low	Resolved
5	Single-Step Ownership Transfer Allows Permanent Loss of Contract Control	Informational	Acknowledged
6	Excessively Lenient Oracle Staleness Threshold Accepts Outdated Prices	Informational	Resolved
7	Missing Pending-Request Guard Allows Overwriting Redemption Requests	Informational	Resolved



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ **Missing Zero Address Validation**

✓ **Private modifier**

✓ **Revert/require functions**

✓ **Multiple Sends**

✓ **Using suicide**

✓ **Using delegatecall**

✓ **Upgradeable safety**

✓ **Using throw**

✓ **Using inline assembly**

✓ **Style guide violation**

✓ **Unsafe type inference**

✓ **Implicit visibility level**

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Medium Severity Issues

Chainlink Oracle Returns Incorrect Price When Circuit Breaker Bounds Are Hit

Resolved**Path**

LibertyICO.sol

Function Name`getLatestPrice()`**Description**

The `getLatestPrice()` function calls Chainlink's `latestRoundData()` and validates staleness, round completeness, and a positive price but never checks whether the returned price is within the aggregator's `minAnswer` / `maxAnswer` bounds. Chainlink aggregators have a built-in circuit breaker: when an asset's real market price crashes below `minAnswer` (or spikes above `maxAnswer`), the aggregator clamps and returns the boundary value instead of the actual price. The feed continues operating and updating `updatedAt` normally, so all your existing staleness and round checks pass cleanly but the price being used is wrong.

Impact

High

Likelihood

Low

RecommendationAdd `minAnswer` / `maxAnswer` bounds checking inside `getLatestPrice()`

Low Severity Issues

Missing Blacklist Mechanism for Restricted Addresses

Resolved

Path

LibertyICO.sol

Function Name

`libertyCoinConversion(), requestRedemption(), executeReturn()`

Description

The LibertyOracle contract has no on-chain blacklist mechanism to block restricted, sanctioned, or malicious addresses from interacting with core protocol functions. Any address can call `requestRedemption()` to initiate physical coin redemption, participate in `libertyCoinConversion()` to mint NFTs by burning LibertyCoin, and receive NFTs via `executeReturn()` as a destination address. The only access gate is the off-chain signer for `libertyCoinConversion()` but `requestRedemption()` and `executeReturn()` have zero caller restrictions beyond ownership checks. Even for the signer-protected path, if the signer key is compromised or a signature is issued before an address is later flagged, there is no on-chain enforcement fallback.

Impact

Medium

Likelihood

Low

Recommendation

Add a blacklisted check across all user-facing entry points



ETH Silently Absorbed on ERC20 Payment in buyToken()**Resolved****Path**

LibertyICO.sol

Function Name**buyToken()****Description**

The buyToken() function is marked payable, meaning it accepts ETH for all payment types including ERC20 token payments. When a user calls buyToken() with paymentType != 0 (an ERC20 token like USDT), any ETH sent alongside the transaction via msg.value is silently accepted and locked inside the contract. The function does not validate that msg.value == 0 on the ERC20 branch. The ERC20 transfer proceeds normally, tokens are delivered to the recipient, and the user receives no indication that their ETH was also consumed. The only recovery path is admin intervention via recoverETH(), which requires the user to trust the admin and wait for manual action.

Impact

Medium

Likelihood

Low

Recommendation

check msg.value is zero or not



Missing Expiry (Deadline) in Signature Verification

Resolved

Path

LibertyICO.sol

Function Name

`verifySign() / PURCHASE_TYPEHASH`

Description

The contract uses EIP-712 structured signatures to authorize token purchases. However, the signed payload defined in PURCHASE_TYPEHASH does not include a deadline or expiry timestamp:

```
bytes32 private constant PURCHASE_TYPEHASH =  
    keccak256(  
        "Purchase(uint256 assetType,address recipient,address caller,uint256  
amount,uint256 nonce)"  
    );
```

As a result, any valid signature issued by the signer remains usable indefinitely, provided:

- The nonce has not been used
- The signer address has not been rotated

This means a user who receives a signature during a promotional pricing period can withhold it and submit it later after tokenAmountPerUSD has been updated to a less favorable rate – to still claim tokens at the old price.

Impact

Medium

Likelihood

Low

Recommendation

Add a deadline field to the EIP-712 typehash and validate it inside verifySign()



Informational Issues

Single-Step Ownership Transfer Allows Permanent Loss of Contract Control

Acknowledged

Path

LibertyOracle.sol

Description

The LibertyOracle contract inherits OpenZeppelin's standard Ownable, which implements single-step ownership transfer via `transferOwnership()`. In this pattern, the new owner address takes effect immediately in a single transaction with no confirmation required from the recipient. There is no pending owner state, no acceptance step, and no way to reverse the transfer once submitted. If the new owner address is mistyped, is a contract that cannot call owner functions, or is otherwise incorrect, ownership and all `onlyOwner` / `onlyOracle` privileges are permanently and irrecoverably lost with no on-chain remedy.

Impact

Low

Likelihood

Low

Recommendation

Replace Ownable with OpenZeppelin's Ownable2Step

Liberty Team's Note

Utmost care should be taken on transfer of ownership. Multisig is not mentioned in the requirement so we have not planned on keeping one. Multisig and/or Timelock can be integrated if initiated by the client.



Excessively Lenient Oracle Staleness Threshold Accepts Outdated Prices

Resolved

Path

LibertyICO.sol

Function Name

`getLatestPrice(), setPriceStaleThreshold()`

Description

The contract considers oracle prices valid for up to 172800 seconds (2 days) via `PRICE_STALE_THRESHOLD`. This is much larger than a typical production safety window for volatile assets. As a result, `buyToken()` may continue using significantly outdated Chainlink prices when market prices have already moved materially. The inline comment also indicates this value was chosen "for testing purposes," which suggests a test configuration may have been left in the production contract logic.

Impact

Low

Likelihood

Low

Recommendation

Reduce the default stale threshold to a production-safe window, typically around 1 hour or less depending on the supported assets



Missing pending-request guard lets a later owner overwrite an existing redemption request

Resolved

Path

LibertyOracle.sol

Function Name

`requestRedemption`

Description

`requestRedemption()` checks `nft.ownerOf(tokenId) == msg.sender`, but it does not check `redemptionRequested[tokenId]`. Because of that, an NFT owner can submit a redemption request, transfer the NFT to another wallet, and the new owner can call `requestRedemption()` again for the same `tokenId`.

The second call overwrites:

- `redemptionRequester[tokenId]`
- the effective pending request identity for that token

Later, `executeRedemption()` trusts the latest stored requester and locks the NFT using that overwritten value. This means the earlier request is silently replaced rather than rejected or explicitly canceled.

Impact

Low

Likelihood

Low

Recommendation

Add a guard in `requestRedemption()` to reject duplicate and adding a `cancelRedemptionRequest()` function for the current requester/current owner

Liberty Team's Comment

We are planning to keep this validation in application because even if it refreshes on call there won't be a big impact on core functioning of contract as the same data will be overridden which will in most cases be mitigated by the application validation.



Functional Tests

Some of the tests performed are mentioned below:

- ✓ Verified deployment and initial token supply allocation.
- ✓ Verified owner-only minting and supply cap enforcement.
- ✓ Verified token burn behavior and balance updates.
- ✓ Verified owner-only operator assignment and base URI updates.
- ✓ Verified operator-only mint and burn functionality.
- ✓ Verified soulbound transfer restrictions and approval controls.
- ✓ Verified token purchase flow using ETH and supported ERC20 payment tokens.
- ✓ Verified Chainlink-based token amount calculation and signer authorization.
- ✓ Verified pause/unpause behavior and admin recovery/configuration functions.
- ✓ Verified LibertyCoin-to-NFT conversion with signature validation and physical asset ID linkage.
- ✓ Verified redemption and return workflows, including custody-state transitions.
- ✓ Verified vault attestation, physical asset relinking/swapping, and burn-for-destroyed-physical flows.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
IERC20 (OpenZeppelin)	Interface	Defines ERC20 interface for tokenAddress (LibertyCoin sold), payment tokens (USDT etc.), and interactions in LibertyICO and LibertyOracle	Correct interface definition matching ERC20 standard	NA
SafeERC20 (OpenZeppelin)	Library	Wraps all ERC20 transfer calls (safeTransfer, safeTransferFrom) in LibertyICO for payment tokens and LibertyCoin distribution	Library correctly handles return values and reverts on failure	If library has a bug, token transfers may silently fail
ERC20 / ERC20Burnable (OpenZeppelin)	Base Contract	LibertyCoin inherits standard fungible token logic and burn capability (burnFrom used by LibertyOracle during NFT conversion)	Correct ERC20 and burn logic; burnFrom respects allowances	Allowance-based burn can be front-run; incorrect decimal handling breaks pricing



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ERC721 (OpenZeppelin)	Base Contract	LibertyNft inherits NFT minting, burning, and transfer logic; _update hook overridden to enforce soulbound restrictions	Standard ERC721 transfer logic is correct and hookable	Override errors could silently allow unauthorized transfers
Ownable (OpenZeppelin)	Base Contract	Single-owner access control in LibertyCoin (minting), LibertyNft (operator/URI config)	Owner is trusted and competent; single-step ownership transfer	Ownership transferred to wrong address causes permanent loss of admin control
AccessControl (OpenZeppelin)	Base Contract	Role-based access (ADMIN_ROLE) in LibertyICO for all admin functions (pause, recover, price updates, signer changes)	Role assignment logic is correct; admin key is secure	Granting ADMIN_ROLE to a malicious address allows fund drainage and parameter manipulation
Pausable (OpenZeppelin)	Base Contract	Emergency circuit-breaker on buyToken (LibertyICO) and most state-changing functions in LibertyOracle	Pause/unpause is correctly gated to authorized roles	If paused maliciously or indefinitely, users cannot purchase or redeem NFTs



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Reentrancy Guard (OpenZeppelin)	Base Contract	Prevents reentrancy on buyToken, recoverETH, recoverToken (LibertyICO) and libertyCoinConversion (LibertyOracle)	Mutex logic is correctly implemented	NA
ECDSA (OpenZeppelin)	Library	Recovers signer address from EIP-712 signatures in verifySign (LibertyICO) and libertyCoinConversion (LibertyOracle)	Recovery returns correct address; no signature malleability	Compromised signer key allows unlimited unauthorized purchases or NFT mints
EIP712 (OpenZeppelin)	Base Contract	Provides typed structured data hashing (_hashTypedData V4) for purchase and conversion authorization flows	Domain separator is correctly computed and unique per contract	Incorrect domain parameters allow cross-contract signature replay
AggregatorV3Interface (Chainlink)	External Interface	Fetches real-time ETH/USD and USDT/USD prices to compute LibertyCoin amounts in getLatestPrice	Oracle returns accurate, timely, and untampered price data	Stale, manipulated, or deprecated feeds cause mispriced token sales; staleness threshold (PRICE_STALE_THRESHOLD) is admin-mutable



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
LibertyCoin (internal)	Contract	Burned via burnFrom in LibertyOracle during NFT conversion; balance checked before conversion	burnFrom correctly enforces allowances; 9-decimal precision matches pricing math	Incorrect allowance causes conversion revert; decimal mismatch with payment tokens breaks getToken calculation
LibertyNft (internal)	Contract	Minted/burned by LibertyOracle (as operator); NFT transfers used for lock/unlock redemption flows	Operator is correctly set to LibertyOracle; _update hook correctly restricts transfers	Operator not set or set incorrectly disables all NFT minting, burning, and locking
Off-chain Signer	Trusted EOA/Service	Authorizes every buyToken call (LibertyICO) and libertyCoinConversion call (LibertyOracle) via EIP-712 signatures	Signer key is secure, available, and issues signatures only for legitimate requests	Key compromise enables unlimited buys/mints; key loss halts all purchases and conversions

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
LibertyCoin.sol	constructor(initialOwner)	What this function can do Mint 250,000 LC to initialOwner; set owner, name, symbol, decimals What this function cannot/should not do Mint beyond MAX_SUPPLY; set zero address as owner; called again after deploy Main invariant(s) totalSupply <= MAX_SUPPLY always Access level Implicit (once at deployment)
LibertyCoin.sol	mint(to, amount)	What this function can do Mint additional LC up to MAX_SUPPLY What this function cannot/should not do Exceed MAX_SUPPLY; mint to zero address; called by non-owner Main invariant(s) totalSupply <= MAX_SUPPLY always Access level onlyOwner

Contract Name	Function	Category
LibertyCoin.sol	burn(amount)	What this function can do Caller burns their own tokens What this function cannot/should not do Burn more than caller balance Main invariant(s) totalSupply decreases by exact burned amount Access level Public
LibertyCoin.sol	burnFrom(account, amount)	What this function can do Burn tokens from account using allowance What this function cannot/should not do Burn beyond allowance or balance Main invariant(s) Allowance and totalSupply decrease correctly Access level Public (requires allowance)
LibertyNft.sol	constructor(initialOwner, _baseURI)	What this function can do Deploy ERC721; set owner, base URI; nextTokenId starts at 1 What this function cannot/should not do Empty base URI; zero owner address Main invariant(s) nextTokenId starts at 1; operator unset until configured Access level Implicit (once at deployment)

Contract Name	Function	Category
LibertyNft.sol	setOperator(_operator)	What this function can do Set the contract allowed to mint/burn NFTs What this function cannot/should not do Set zero address; clear stale approvals from old operator Main invariant(s) Only one active operator at a time Issues Stale approve()/setApprovalForAll() records from old operator persist after rotation – latent authorization drift Access level onlyOwner
LibertyNft.sol	approve(to, tokenId)	What this function can do Approve configured operator for a specific token What this function cannot/should not do Approve any address other than configured operator Main invariant(s) No marketplace approvals possible Access level Public (owner only; operator target only)
LibertyNft.sol	setApprovalForAll	What this function can do Grant full approval to configured operator only; revoke for any address What this function cannot/should not do Grant approval to non-operator address Main invariant(s) No third-party marketplace approvals possible Access level Public (restricted target)



Contract Name	Function	Category
LibertyICO.sol	constructor	What this function can do Init ICO with owner, signer, sale token; set default ETH + USDT payment configs What this function cannot/should not do Zero address for owner/signer/token Main invariant(s) ADMIN_ROLE granted to owner; ETH and USDT active by default Access level Implicit (once at deployment)
LibertyICO.sol	buyToken(recipient, paymentType, tokenAmount, sign)	What this function can do Accept ETH/ERC20, verify signature, compute LC via oracle, transfer LC to recipient What this function cannot/should not do Reuse nonce; accept ETH for ERC20 type; send to zero address; operate while paused Main invariant(s) Each nonce used once; LC out equals oracle-computed amount Issue USDT blacklisting on contract or recovery wallet bricks USDT path external dependency risk, not a code bug Access level Public (valid signer signature required)

Contract Name	Function	Category
LibertyICO.sol	getLatestPrice(paymentType)	What this function can do Fetch and validate Chainlink price; revert on stale/zero/incomplete round What this function cannot/should not do Return stale or zero price; modify state Main invariant(s) Price always positive and within PRICE_STALE_THRESHOLD Issue Oracle decimals hardcoded as 1e8 non-standard feeds with different precision silently produce wrong LC amounts Access level Public
LibertyICO.sol	getToken(paymentType, tokenAmount)	What this function can do Compute LC output using oracle price and tokenAmountPerUSD; uses Math.mulDiv to prevent overflow What this function cannot/should not do Return zero; accept zero input; modify state Main invariant(s) Output always positive for valid inputs Access level Public
LibertyICO.sol	recoverETH(walletAddress)	What this function can do Withdraw full ETH balance to specified wallet What this function cannot/should not do Send to zero address; withdraw zero balance Main invariant(s) Contract ETH balance zero after call Access level onlyRole(ADMIN_ROLE), nonReentrant



Contract Name	Function	Category
LibertyICO.sol	recoverToken(_token, wallet, amount)	What this function can do Withdraw specified ERC20 amount to wallet What this function cannot/should not do Withdraw beyond balance; send to zero address Main invariant(s) Contract balance decreases by exact amount Issue USDT-blacklisted recovery wallet causes revert – funds stuck until different wallet used Access level onlyRole(ADMIN_ROLE), nonReentrant
LibertyICO.sol	setPaymentTo kenDetails	What this function can do Add/update payment method config – feed, token address, decimals, status What this function cannot/should not do Zero price feed when active; invalid decimals; non-contract ERC20 address Main invariant(s) ETH type always zero token address and 18 decimals Access level onlyRole(ADMIN_ROLE)
LibertyICO.sol	setTokenPrice PerUSD	What this function can do Update LC units issued per 1 USD What this function cannot/should not do Set to zero; called by non-admin Main invariant(s) tokenAmountPerUSD > 0 always; affects all future getToken immediately Access level onlyRole(ADMIN_ROLE)



Contract Name	Function	Category
LibertyICO.sol	setPriceStale Threshold	What this function can do Update max acceptable oracle data age in seconds What this function cannot/should not do Set to zero; set same value; called by non-admin Main invariant(s) PRICE_STALE_THRESHOLD > 0 always Access level onlyRole(ADMIN_ROLE)
LibertyICO.sol	receive()	What this function can do Reject all direct ETH transfers What this function cannot/should not do Accept ETH outside buyToken Main invariant(s) ETH enters only via buyToken Access level External, always reverts
LibertyICO.sol	fallback()	What this function can do Reject all calls to non-existent functions What this function cannot/should not do Execute any logic silently Main invariant(s) No unrecognized selector executes Access level External, always reverts



Contract Name	Function	Category
LibertyOracle.sol	constructor	What this function can do Init with owner, signer, vault, NFT and LC contract references What this function cannot/should not do Zero address for NFT or LibertyCoin Main invariant(s) nft and libertyCoin immutable after deploy Access level Implicit (once at deployment)
LibertyOracle.sol	setCoinsPerNft(newAmount)	What this function can do Update LC amount required per NFT conversion What this function cannot/should not do Set to zero; called by non-oracle Main invariant(s) coinsPerNft > 0 always Issue Not committed in EIP-712 signature oracle can raise price between signing and execution; user burns more LC than agreed with no on-chain slippage protection Access level onlyOracle
LibertyOracle.sol	requestRedemption(tokenIds[])	What this function can do NFT owner signals intent to redeem physical coin; sets request flags What this function cannot/should not do Called by non-owner; request on destroyed/locked tokens Main invariant(s) Token stays with owner until oracle executes Access level Public (caller must be token owner)



Contract Name	Function	Category
LibertyOracle.sol	executeRedemption(tokenIds[])	What this function can do Lock NFTs into contract after validating prior request; records lockedOwner What this function cannot/should not do Execute without prior request; lock destroyed tokens; called by non-oracle Main invariant(s) locked[tokenId] == true; lockedOwner correctly set Access level onlyOracle, whenNotPaused
LibertyOracle.sol	requestReturn(tokenIds[])	What this function can do lockedOwner signals intent to return physical coin and reclaim NFT What this function cannot/should not do Called by non-lockedOwner; request on unlocked/destroyed tokens Main invariant(s) Token stays locked until oracle executes Access level Public (caller must be lockedOwner)
LibertyOracle.sol	executeReturn(tokenIds[], sendToVault)	What this function can do Unlock NFTs; send to lockedOwner or vault; clear all lock/request state What this function cannot/should not do Execute without prior request; return destroyed tokens; send to vault if vault is zero Main invariant(s) locked[tokenId] == false; all flags cleared; NFT correctly transferred Access level onlyOracle, whenNotPaused



Contract Name	Function	Category
LibertyOracle.sol	libertyCoinConversion(numberOfNFTs, physicalAssetIds[], sign)	What this function can do Burn coinsPerNft × numberOfNFTs LC; mint NFTs; link each to physical ID; set custody to LockedInVault What this function cannot/should not do Proceed with insufficient balance/allowance; reuse nonce; mismatched array length; operate while paused Main invariant(s) LC burned = coinsPerNft × numberOfNFTs; each NFT linked to unique physical ID; nonce used once Issue coinsPerNft excluded from signed payload – race condition allows oracle to inflate burn cost after signature issuance. PoC: sig issued at 250e9 → oracle sets 500e9 → user burns 2× expected Access level Public (valid signature required), nonReentrant, whenNotPaused
LibertyOracle.sol	burnForDestroyedPhysical(tokenIds[])	What this function can do Mark coins destroyed; clear all protocol state; burn NFTs What this function cannot/should not do Burn already-destroyed tokens; exceed batch limit; called by non-oracle Main invariant(s) destroyedPhysical[tokenId] == true; NFT gone; all mappings cleared Access level onlyOracle, whenNotPaused



Contract Name	Function	Category
LibertyOracle.sol	computeOnchainCustodyState(tokenId)	What this function can do Derive custody state from live storage flags – does not rely solely on stored custodyState What this function cannot/should not do Modify state; contradict actual lock/destroy/request flags Main invariant(s) Destroyed → Destroyed; locked → RedeemedOut; non-existent → Unknown Access level Public



3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
LibertyICO.sol	ETH	ETH received from token purchases via buyToken() held until admin calls recoverETH()
LibertyICO.sol	ERC20 (LibertyCoin)	LC tokens held for sale distributed to buyers; remainder recoverable via recoverToken()
LibertyICO.sol	ERC20 (USDT + other payment tokens)	Payment tokens received from buyers held until admin calls recoverToken()
LibertyOracle.sol	ERC721 (LibertyNft)	NFTs locked during physical redemption flow held between executeRedemption() and executeReturn()
LibertyCoin.sol	ERC20 (LC)	No direct custody token balances live in user wallets; burnFrom reduces supply on conversion
LibertyNft.sol	ERC721 (LBT)	No direct custody NFTs held by users or LibertyOracle; contract only manages minting/ burning logic



3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
LibertyICO.sol	buyToken() (ETH path)	ETH → contract	LC → recipient	Public	Oracle price manipulation; stale feed causes mispriced LC; hardcoded 1e8 decimals breaks non-standard feeds
LibertyICO.sol	buyToken() (ERC20 path)	USDT/ ERC20 → contract	LC → recipient	Public	USDT blacklist bricks payment path; fee-on-transfer tokens break 1:1 accounting; no slippage protection
LibertyICO.sol	recoverETH()	—	ETH → admin wallet	ADMIN_ROLE	Compromised admin drains all ETH; no timelock or multi-sig enforced on-chain
LibertyICO.sol	recoverToken()	—	ERC20 → admin wallet	ADMIN_ROLE	Compromised admin drains all tokens including unsold LC; USDT-blacklisted wallet causes stuck funds
LibertyICO.sol	libertyCoinConversion()	LC burned from caller	NFT → caller	Public (+ valid sig)	coinsPerNft not in signed payload — oracle can inflate burn cost after sig issuance; no max-cost guard for user



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
LibertyOracle.sol	executeRedemption()	NFT → contract (locked)	–	onlyOracle	Oracle locks NFT without user re-confirming; if oracle is compromised, NFTs locked indefinitely
LibertyOracle.sol	executeReturn()	–	NFT → lockedOwner or vault	onlyOracle	Compromised oracle redirects NFT to vault instead of rightful owner; sendToVault flag unilaterally controlled by oracle
LibertyOracle.sol	burnForDestroyedPhysical()	NFT burned (destroyed)	–	onlyOracle	Compromised oracle permanently destroys user NFTs with no recovery path; irreversible action
LibertyCoin.sol	mint()	–	LC → any address	onlyOwner	Compromised owner mints up to MAX_SUPPLY; dilutes all holders; no timelock or cap-per-tx guard
LibertyCoin.sol	burnFrom()	LC burned from account	–	Public (+ allowance)	Excessive allowance approval allows LibertyOracle to burn more than expected if coinsPerNft is raised



Closing Summary

In this report, we have considered the security of the Liberty Protocol. We performed our audit according to the procedure described above.

No critical or high-severity issues were identified in the Liberty Protocol. One medium, three low, and three informational issues were found, all of which have been resolved and acknowledged by the Liberty Protocol team

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

April 2026

For



REAL LIBERTY COPPER



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com